BACSE202 · Fall Semester 2026

PolyVote

A Multi-Tenant, Ranked-Choice Election Platform with a Tamper-Evident Audit Trail

Reviews 2 and 3 · Final Project Review · Guide: Dr. Katari Balakrishna



S.No	Name	Register No
1	Tharakesharasu Thennarasu	25BCE2699
2	Pawan Durga Karthik	25BCE2712
3	Shivam Malge	25BCE0386

REVIEW 2 · WHERE WE STAND

What we submitted, what we built, what changed

SUBMITTED

Review 1

Design, relational schema and ER diagram, security approach, literature review, and a planned architecture on Oracle, MongoDB and Streamlit.

BUILT

Since Review 1

A working system: React voter booth and admin console, FastAPI service, SQLite schema with trigger and views, instant-runoff tally, hash-chained receipts and a live tamper test.

CHANGED

And why

SQLite stands in for Oracle and a JSON document ledger for MongoDB; React + FastAPI now sits beside Streamlit; an ADMIN_USERS table was added.

75%

of the Review 1 scope

Left for the final review: Oracle with PL/SQL procedures and DCL, MongoDB Atlas, cloud deployment, and the POSITION entity.

Next: the design recap, updated diagrams, then the running system and its outputs.

SYSTEM RATIONALE

PolyVote tackles three weaknesses in digital elections

PROBLEM

Plurality & Insider Risk

Plurality records one choice and can split support across three or more candidates, producing a winner without majority backing. Single-election systems are hard to reuse across organizations, and an administrator with `UPDATE` access can silently alter results.

SOLUTION

Ranked Choice & Hash Chain

Let voters rank candidates, then use Instant-Runoff Voting (IRV). Host organizations in one tenant-scoped schema; separate voter identity from ballot content; link audit records in a hash chain.

OUTCOME

Reusable Web Service

A reusable, multi-organization web service (cloud hosting is planned for the final review), designed for a continuing-ballot majority winner, detectable record changes, and a secret ballot.

CORE ARCHITECTURE

Six database techniques make the platform work

1 Multi-Tenant Schema

One shared relational schema isolates tenant institutions through foreign-key predicates and row-level policies.

2 Instant-Runoff Tally

Tallies first choices, eliminates lowest candidates iteratively, and redistributes votes until reaching a true majority.

3 SHA-256 Hash Chaining

Every vote event cryptographically references its prior block digest, rendering unauthorized retro-edits instantly invalid.

4 Polyglot Persistence

A relational store keeps structured entities; an append-only JSON event ledger (MongoDB document model) supports audit. Oracle and MongoDB hosting follow.

5 Integrity

UNIQUE and CHECK constraints plus a trigger block duplicate participation independently of application logic; PL/SQL packages follow with Oracle.

6 Cloud Deployment

Runs today as one web service (API and UI). Hosting it on a cloud platform is the remaining deployment step.

THEORETICAL FOUNDATION

Six studies frame PolyVote’s design choices

Ref	Author / Year	Theme	Technique	Tools / Data	Key Finding	Limitation
[1]	Aulbach et al. (2008)	Multi-tenant Data	Schema Mapping	Relational DBs	Shared schema minimizes SaaS operational costs	DB privileges cannot block insider table edits
[2]	Haber & Stornetta (1991)	Audit Trails	Hash Chaining	Timestamps, SHA	Chained block digests establish tamper evidence	Requires external verification monitor
[3]	Sadalage & Fowler (2012)	Polyglot Data	Hybrid SQL/NoSQL	Dual DB Engines	Specialized engines optimize discrete access patterns	Cross-database distributed consistency overhead
[4]	Chaum (2004)	Voter Privacy	Pseudonym Receipts	Crypto Receipts	Decouples eligibility registry from cast ballots	Relies on physical printer hardware verification
[5]	Crosby & Wallach (2009)	Authenticated Logs	Merkle History Trees	Tree-based Hashes	Enables lightweight, verifiable history membership	Tree generation adds computational latency
[6]	Blom et al. (2019)	IRV Auditing	Risk-Limiting Audit	Cast Vote Records	Mathematical audit proofs across elimination rounds	Assumes physical paper ballots for

RESEARCH GAPS & TAMPER MODEL

Four gaps remain between auditability and usable digital voting

G1 · Multi-Tenant Isolation [1]

Shared schemas rely on DBMS privileges; rogue root DBAs can modify election rows undetected without cryptographic ledgers.

G2 · Passive Audit Logs [2, 5]

Hash chains record history, but tamper evidence fails unless active automated verification routines routinely validate digests.

G3 · Paper-Dependent Privacy [4]

Cryptographic receipts traditionally demand specialized in-person printers, proving unusable for fully digital web voting.

G4 · Paper-Reliant IRV Auditing [6]

Risk-limiting audits assume physical ballot boxes for manual recount; purely digital IRV workflows lack native auditability.

PolyVote Hash Chain & Tamper Verification

Each cast ballot generates an append-only JSON record linking its payload to the prior record hash.

Block N-1 · Cast Ballot Receipt
PrevHash: 0x8F4D... | Hash: 0xA19B...

↓ SHA-256 Chaining

Block N · Tokenized Cast Ballot
PrevHash: 0xA19B... | Hash: 0x7E3C...

Tamper Detection Rule: If an admin modifies Block N-1 in MongoDB/Oracle, SHA256(Block N-1) ≠ PrevHash(Block N), immediately alerting audit inspectors.

PolyVote Design Challenge: unifies multi-tenant reuse, automated integrity checks, and tokenized voter privacy.

Four modules separate identity, tallying, and audit

Relational core (SQLite now, Oracle planned): Stores organizations, elections, candidates, voters and admin accounts; constraints, a trigger and views enforce integrity.

Python · tally engine: Counts rankings, eliminates the lowest candidate, redistributes ballots, and handles empty elections and exhausted ballots; last-place ties are flagged.

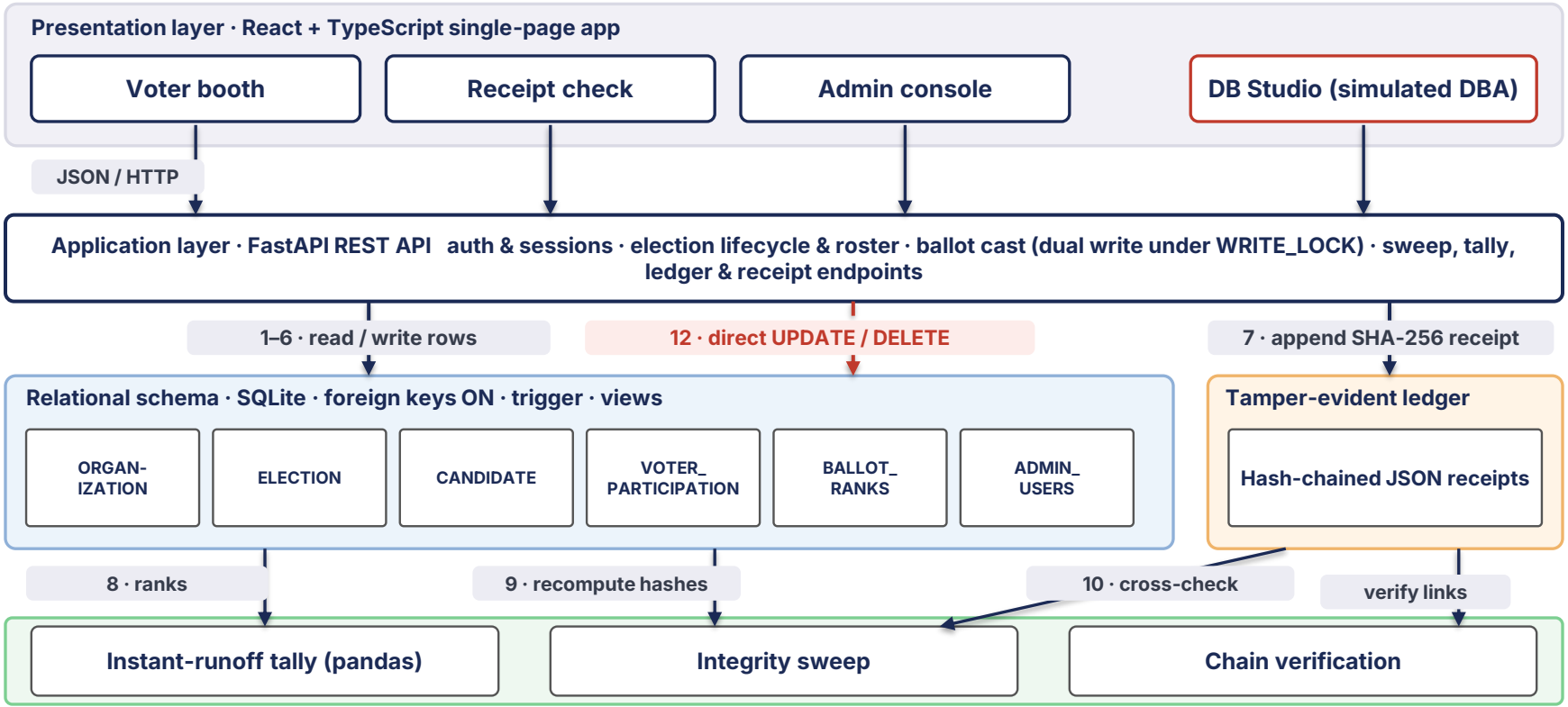
JSON audit ledger (MongoDB planned): Stores one hash-chained receipt per ballot, each linked to its predecessor and re-checked by the integrity sweep.

React + FastAPI · application: Voter booth, receipt check and admin console over a REST API; coordinates the anonymous dual write. Streamlit is retained.

Core entities: ORGANIZATION · ELECTION · CANDIDATE · VOTER_PARTICIPATION · BALLOT_RANKS · ADMIN_USERS (POSITION is deferred). Privacy boundary: VOTER_PARTICIPATION and BALLOT_RANKS remain separate; only an anonymous token connects voter participation to ballot content.

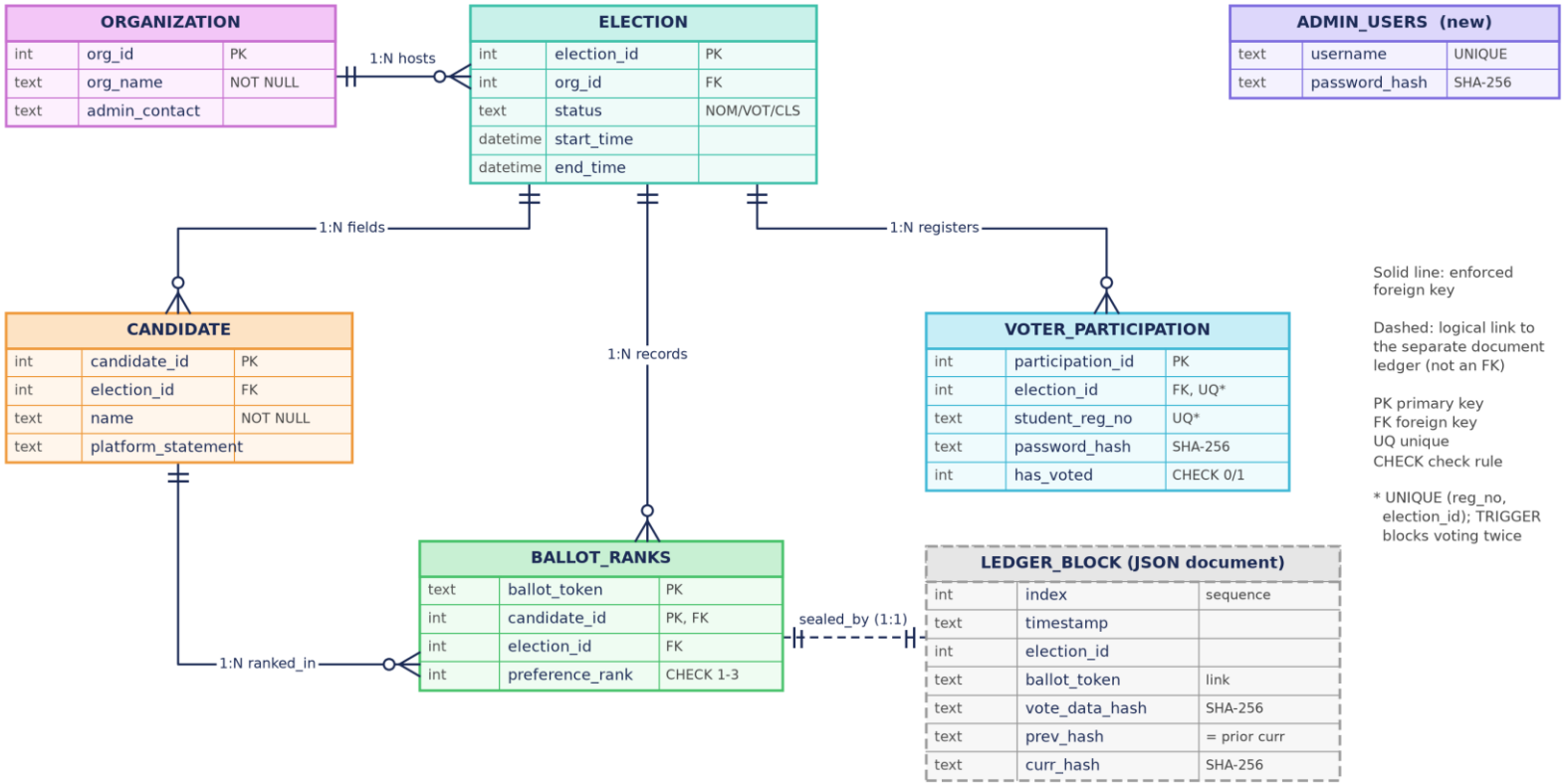
SYSTEM ARCHITECTURE

Data flows through three layers, with the ledger kept apart



ER DIAGRAM & RELATIONAL SCHEMA

Tables and ledger are linked by token, not by identity



DATABASE INTEGRITY

The database itself refuses invalid votes

Rule	Enforced by	Response when we tried to break it
One vote per voter	`TRIGGER trg_block_duplicate_vote`	**DBMS Trigger Violation: Voter has already submitted a ballot.**
Top three ranks only	`CHECK (preference_rank BETWEEN 1 AND 3)`	**CHECK constraint failed: preference_rank BETWEEN 1 AND 3**
No candidate twice on one ballot	`PRIMARY KEY (ballot_token, candidate_id)`	**UNIQUE constraint failed: BALLOT_RANKS.ballot_token, BALLOT_RANKS.candidate_id**
Only real candidates	`FOREIGN KEY → CANDIDATE`	**FOREIGN KEY constraint failed**

Live views

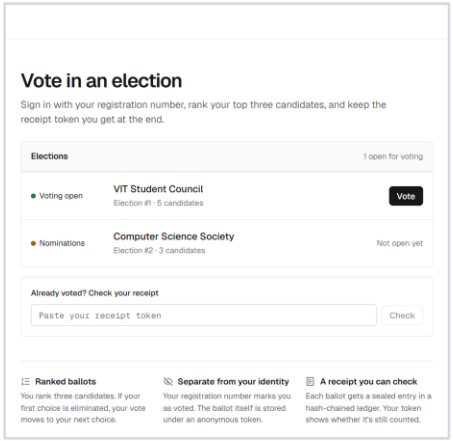
VIEW_ELECTION_TURNOUT and VIEW_FIRST_PREFERENCES feed the dashboards: 45 of 60 voters, 75% turnout.

Each statement ran on the live database

and was rolled back; the messages shown are the database's own, not the application's.

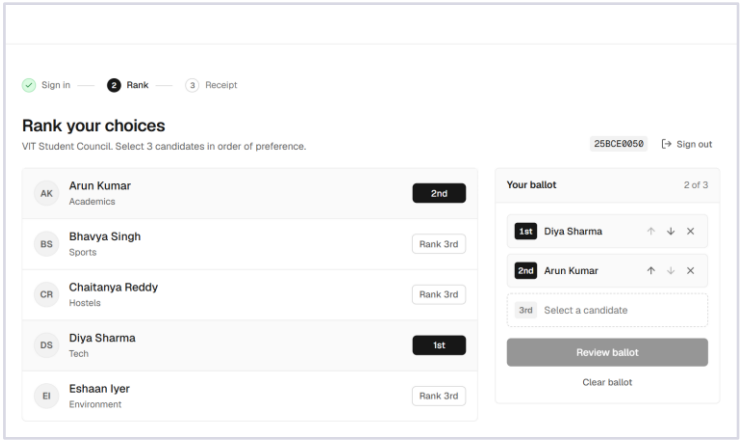
VOTER SIDE

Voting takes three steps and ends with a receipt



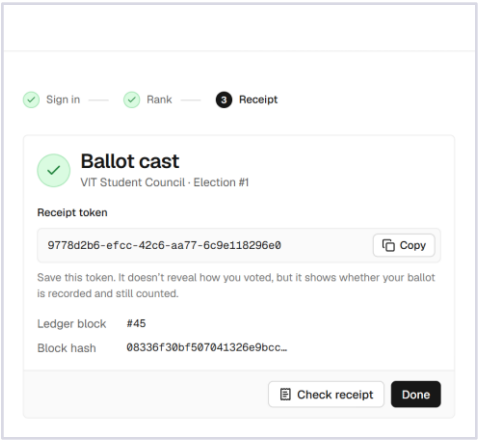
1 · Choose an open election

Only elections in the Voting phase can be entered.



2 · Rank three candidates

Tap in order of preference; review before casting.



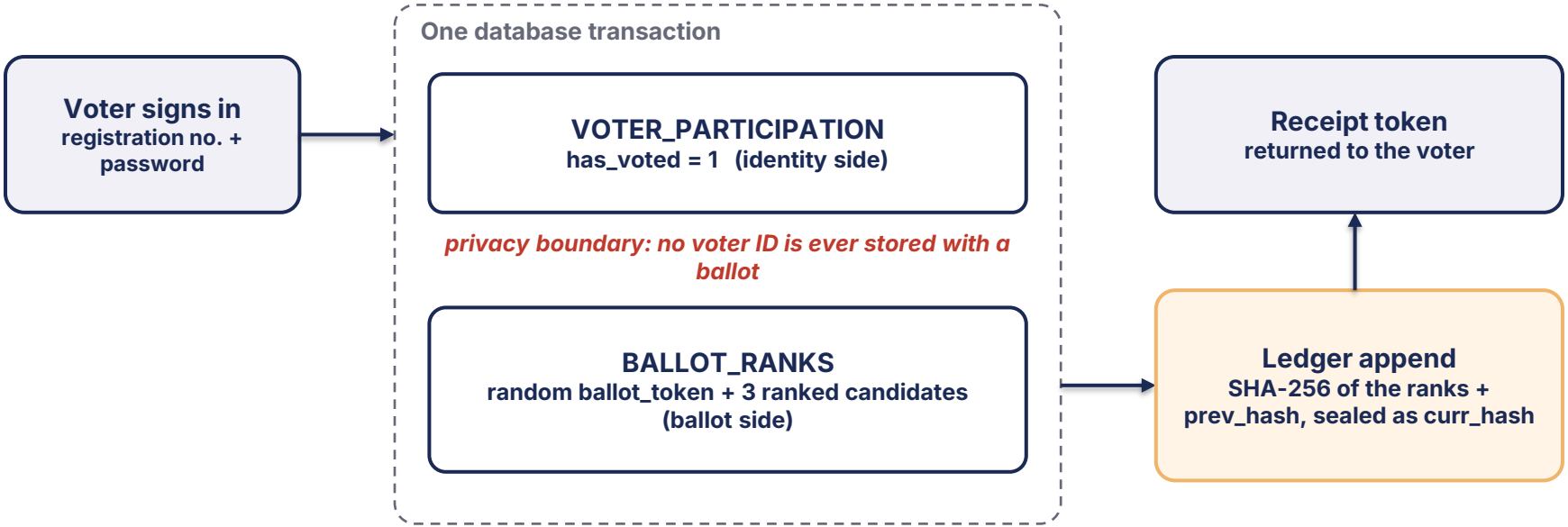
3 · Keep the receipt token

It proves the ballot is sealed, not how you voted.

Guards in the flow: wrong password is rejected, a voter who already voted is refused, and the ballot cannot be changed once cast.

PRIVACY BY DESIGN

One vote becomes two records that cannot be joined



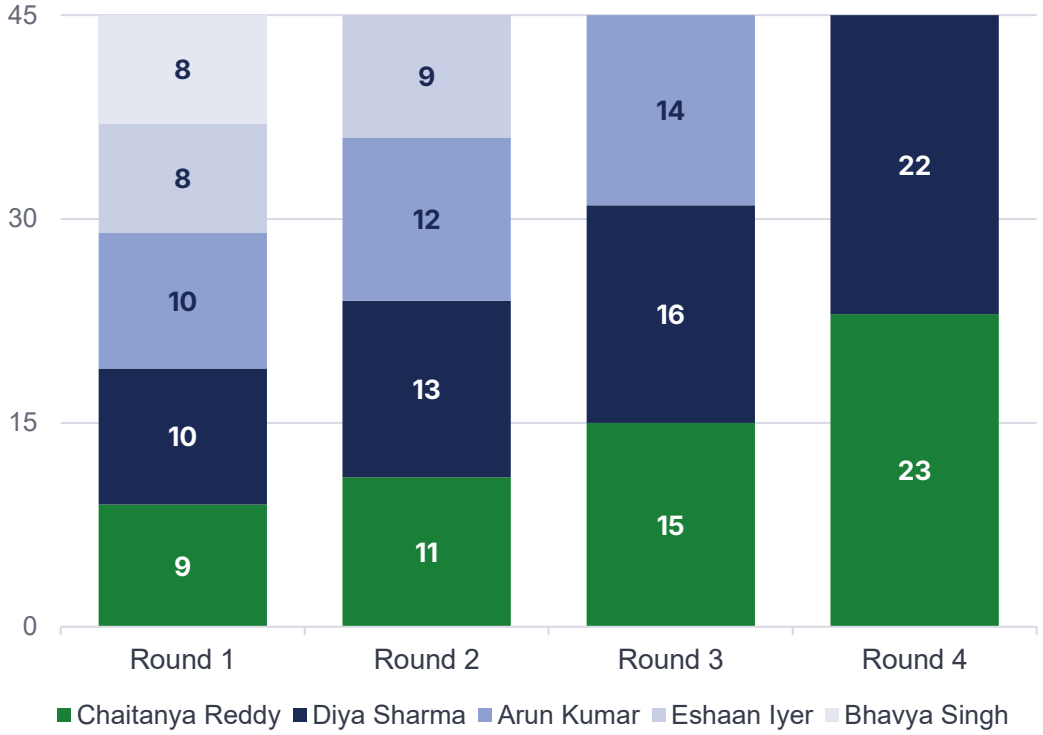
Atomic
The voter flag and the ballot rows commit together, then the receipt is appended.

Serialized
A server-side write lock keeps simultaneous ballots from corrupting the ledger.

Verifiable
Voters can look their token up later without revealing their choices.

TALLYING ENGINE

Instant-runoff counts until someone holds a majority



Chaitanya Reddy wins

23 of 45 continuing ballots (51.1%) in round 4, against 22 for Diya Sharma.

How it runs

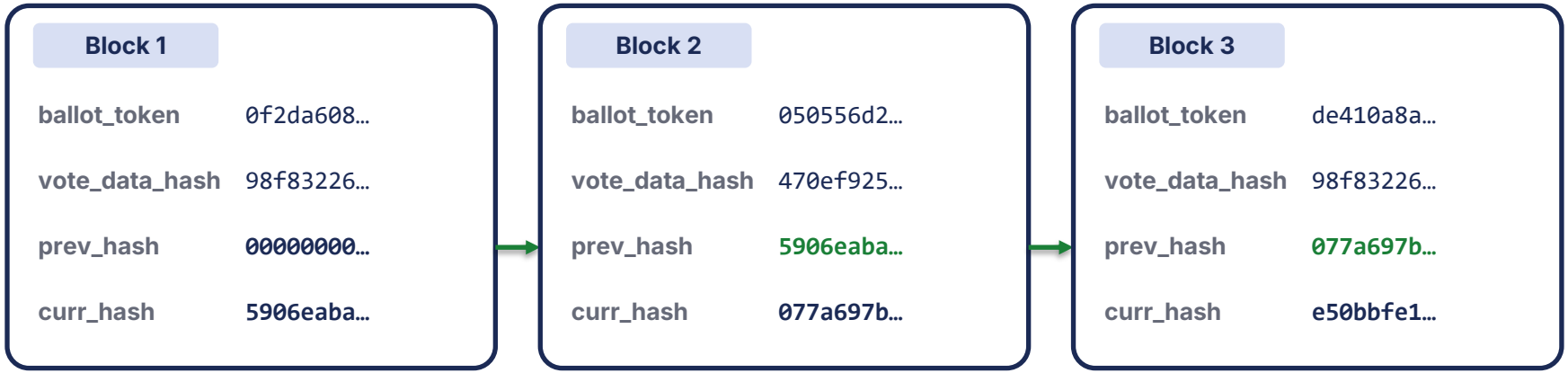
Count each ballot's top continuing choice.

No majority? Eliminate the last-place candidate and move those ballots to their next choice.

Edge cases: exhausted ballots, empty elections, and last-place ties are flagged in the interface.

AUDIT LEDGER

Every ballot is sealed in a hash chain



Block 2's prev_hash equals block 1's curr_hash, and so on: editing any block breaks every link after it.

Seal

vote_data_hash = SHA-256 of the ballot's ranks; curr_hash covers the whole block.

Chain check

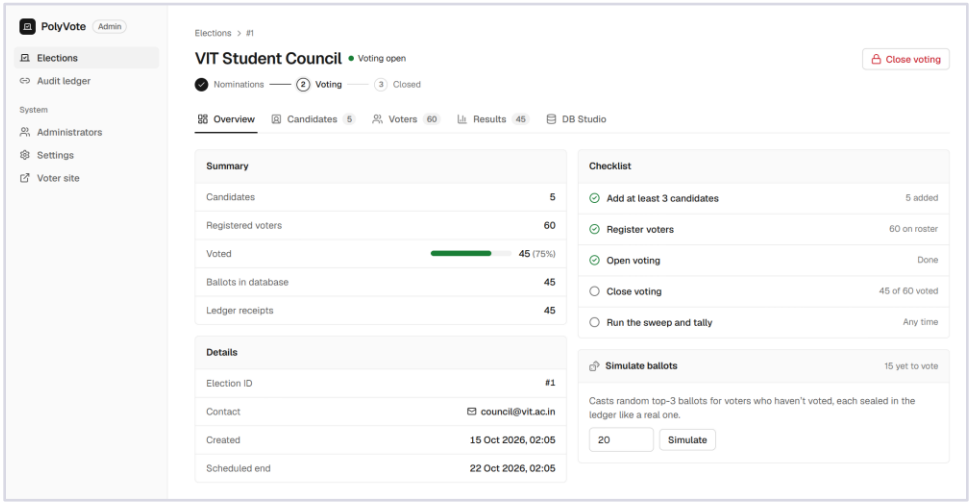
audit_ledger.py recomputes every hash and link: 45 of 45 blocks intact.

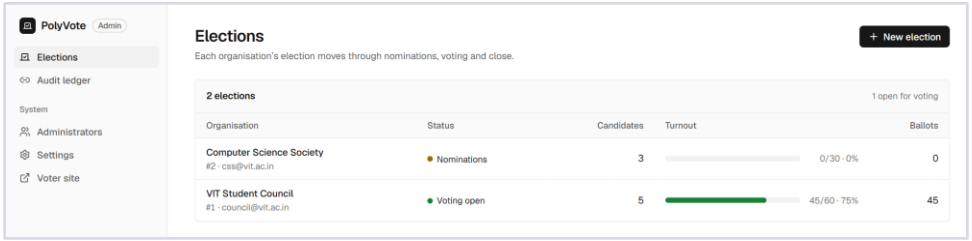
Ballot check

The integrity sweep recomputes each ballot from the database and compares it with its receipt.

ADMINISTRATOR SIDE

Officers run each election through a guarded lifecycle





Election lifecycle

1 Nominations
Add candidates; build the roster by hand or in bulk.

2 Voting
Candidates are locked; voters cast ballots; simulation tools for testing.

3 Closed
No more votes; run the integrity sweep, then count.

Each change asks for confirmation, and a closed election cannot be reopened.

LIVE TAMPER TEST

A rogue DBA edits the database and the sweep catches it

1 Edit

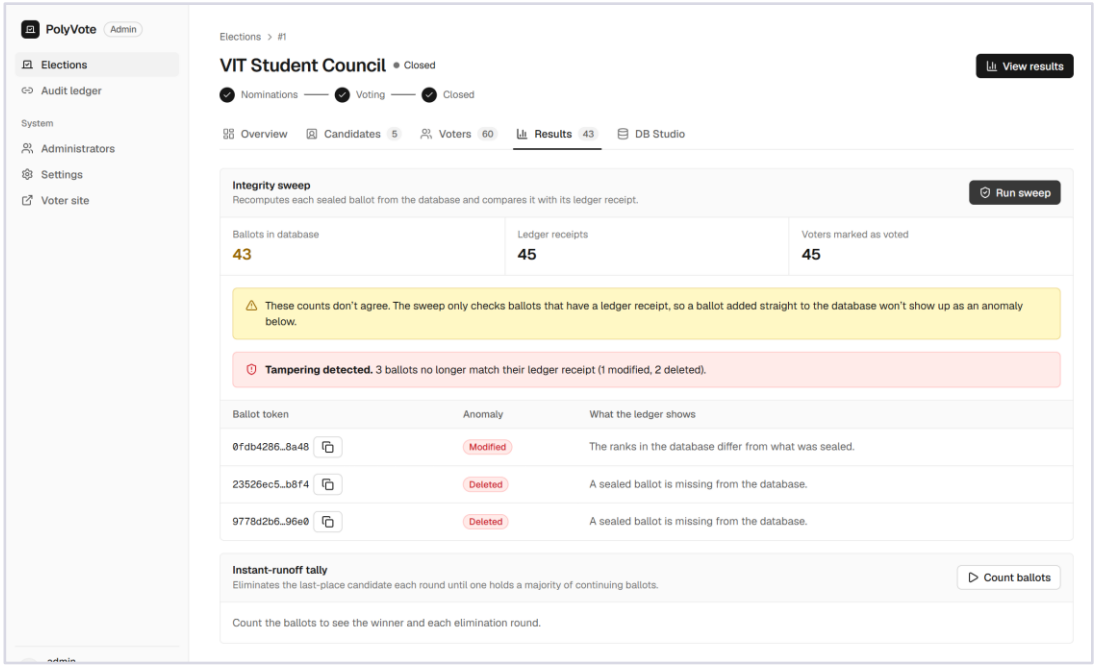
In DB Studio we re-assigned one ballot's first choice.

2 Delete

We deleted two ballot rows, including a voter's own.

3 Sweep

Run integrity sweep: every sealed ballot is recomputed and compared.



Result: 3 of 3 flagged (1 modified, 2 deleted)

VERIFICATION

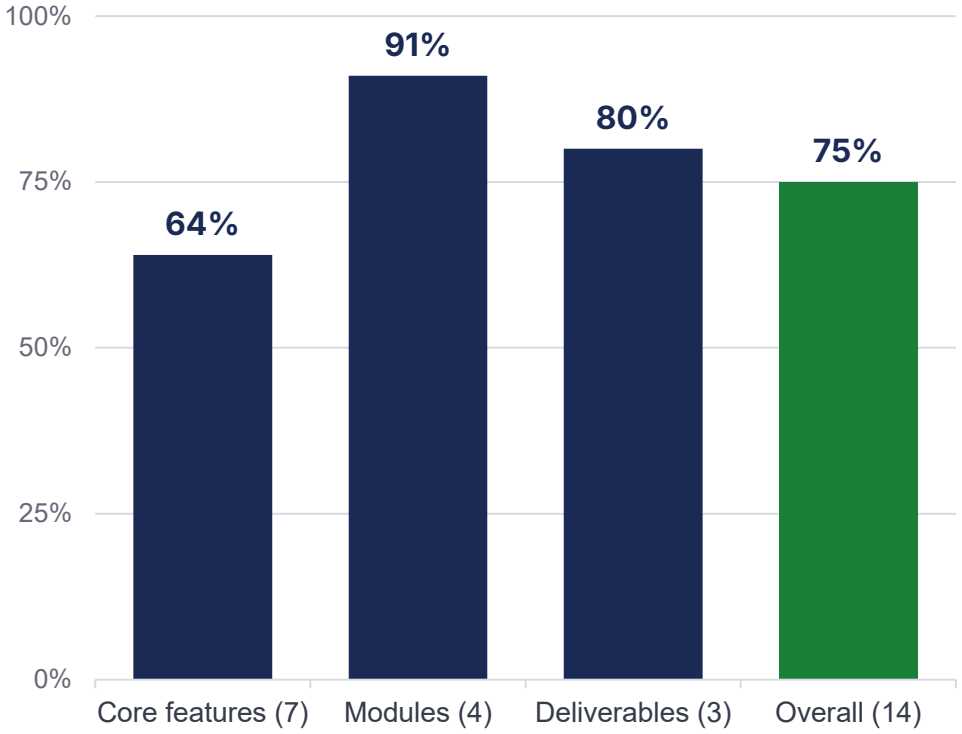
Seven checks, seven expected outcomes

Check	Expected	Observed
Same voter votes twice	Refused	You have already voted in this election.
Wrong password at sign-in	Rejected	Incorrect password.
Ledger hash chain	Intact	Chain intact across all 45 receipts
1 ballot re-ranked, 2 deleted in the database	All three flagged	1 modified and 2 deleted detected
Voter checks a deleted ballot’s receipt	Warned	Recorded, but missing from the count
Counts after tampering	Mismatch visible	43 ballots vs 45 receipts vs 45 voted
Ranked-choice majority	Winner holds a majority	Chaitanya Reddy: 23 of 45 (51.1%)

Outputs captured on a demo election: 60 registered voters, 45 ballots cast, five candidates.

IMPLEMENTATION STATUS

About 75% of the Review 1 scope is implemented



Done

Multi-tenant schema, trigger and views, instant-runoff tally, hash-chained ledger, tamper detection, voter and admin applications.

Remaining for the final review

Oracle with PL/SQL procedures and DCL

MongoDB Atlas for the ledger

Cloud deployment

POSITION entity

PLAN VS BUILD

Where the build differs from the Review 1 plan, and why

Area	Review 1 plan	Built for Review 2	Why
Relational DBMS	Oracle Autonomous DB with PL/SQL	SQLite with trigger, views, constraints	Zero-setup development; schema kept portable
NoSQL ledger	MongoDB Atlas	Hash-chained JSON documents	Same document model without a hosted cluster
Frontend	Streamlit	React + FastAPI; Streamlit kept	Separate voter and admin experiences, mobile support
Schema	Five core tables	+ ADMIN_USERS; POSITION deferred	Officer logins needed a table
Lifecycle	Timestamp-driven procedures	Guarded Nominations → Voting → Closed	SQLite has no stored procedures

Unchanged targets: Oracle with PL/SQL, MongoDB Atlas and cloud deployment remain the goals for the final review; the code is structured so they are a migration, not a rewrite.

ROADMAP & DELIVERABLES

Three reviews move from design to a finished system

REVIEW 1 · DONE

Presented the design, schema, security approach, and planned system architecture.

REVIEW 2 · NOW

Working implementation, about 75% of the planned scope, with live outputs and a tamper demonstration.

FINAL REVIEW

Oracle + PL/SQL, MongoDB Atlas, cloud deployment, POSITION entity, then comparison with existing e-voting systems.

Final review: MongoDB and hosting delivered; Oracle, PL/SQL, DCL roles and POSITION left out, SQLite kept.

Remaining · Application: host the platform live in the cloud
Database: PL/SQL procedures, DCL roles, Oracle and MongoDB scripts
Verification: detect ballots injected without a receipt

PICKING UP FROM REVIEW 2

Review 2 left a short list; two new features lead it

REVIEW 2 ENDED WITH

- About 75% of the scope built and demonstrated.
- Open: MongoDB Atlas, hosting, injected-ballot detection.
- Faculty feedback: confirm the voter with a second factor.
- Deferred on purpose: Oracle with PL/SQL, POSITION entity.

NEW: EMAIL OTP

A correct password is no longer enough.

A 6-digit code is emailed to the voter, valid 5 minutes, one use, 3 guesses.

Answers the Review 2 comment.

NEW: QR VOTING

Scan, sign in, vote.

The vote page shows a QR code for the public link, so any phone opens the ballot in seconds.

Works with the emailed code on the same phone.

RESPONSE TO FEEDBACK

Each Review 2 comment has a concrete answer

Comment at Review 2	What we did	Evidence
Confirm the voter: OTP or face ID	Emailed one-time code after the password; separate verification database; admin outbox	14 of 14 OTP tests pass
Finish MongoDB	Ledger now in MongoDB Atlas, unique-index concurrency control, file fallback, migration script	25 blocks migrated; live health endpoint
Host it online	Public HTTPS link through a tunnel; QR code on the voter page for phones	Cast a ballot from a phone
Prove it is better	47-scenario executable suite run on the Review 2 design and the final design	6 of 13 to 12 of 13 attacks caught
Weak spots we knew about	Injected ballots, forged receipts, rewritten chains, renamed candidates	Each has a test and a fix

Where we say "future work", it is because the design needs something we cannot add honestly in one semester, and we say what.

THREAT MODEL

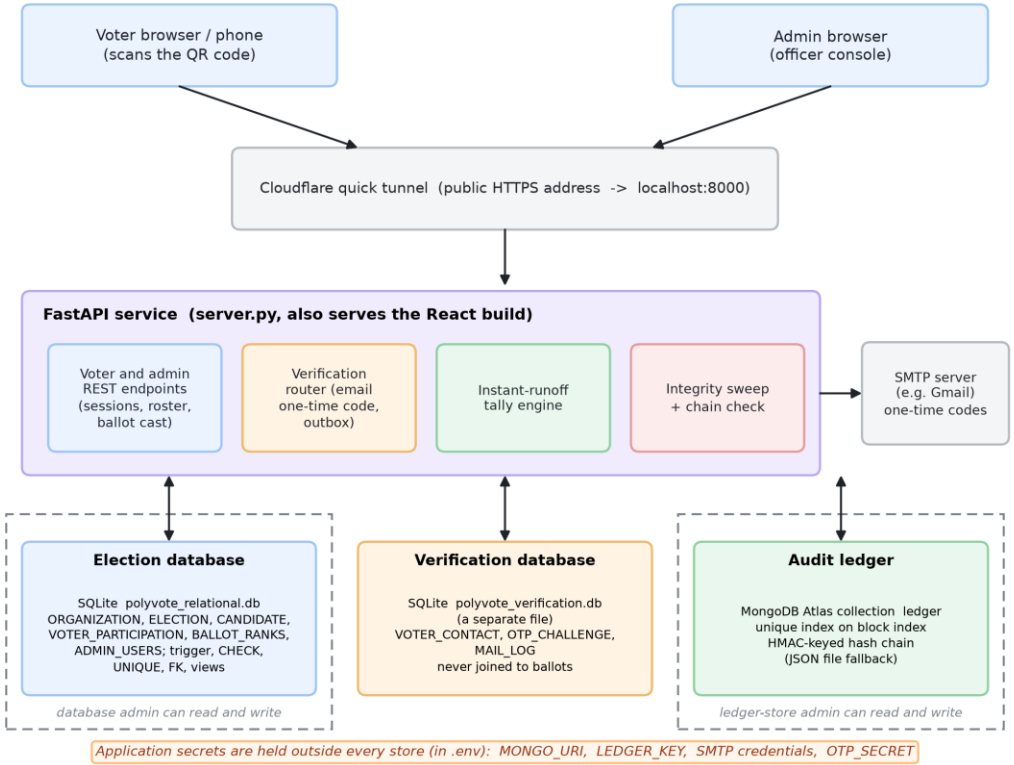
Who the attacker is, and who we do not stop

ID	Adversary	Capability	PolyVote response	Status
A1	Voter with a leaked or guessed password	Signs in as someone else	Emailed one-time code; one-vote trigger	Prevented
A2	Network observer	Reads or alters traffic	Out of scope beyond TLS at the tunnel	Not covered
A3	Rogue database administrator	Read/write on the election database only	Integrity sweep: modified, deleted, injected, mismatch	Detected
A4	Ledger-store administrator	Read/write on the ledger store only	HMAC hash chain; unique block index	Detected
A5	Insider with both stores, no secrets	Edits both stores to match each other	Keyed chain + sweep catch forged receipts, rewrites	Detected
A6	Insider with both stores and the ledger key	Forges valid blocks	Needs external anchoring (future work)	Not covered
A7	Roster injection through the admin path	Adds a ghost voter, votes as them through the API	Needs a committed roster (future work)	Not covered

Detected means found by an auditor running the checks, not blocked in real time.

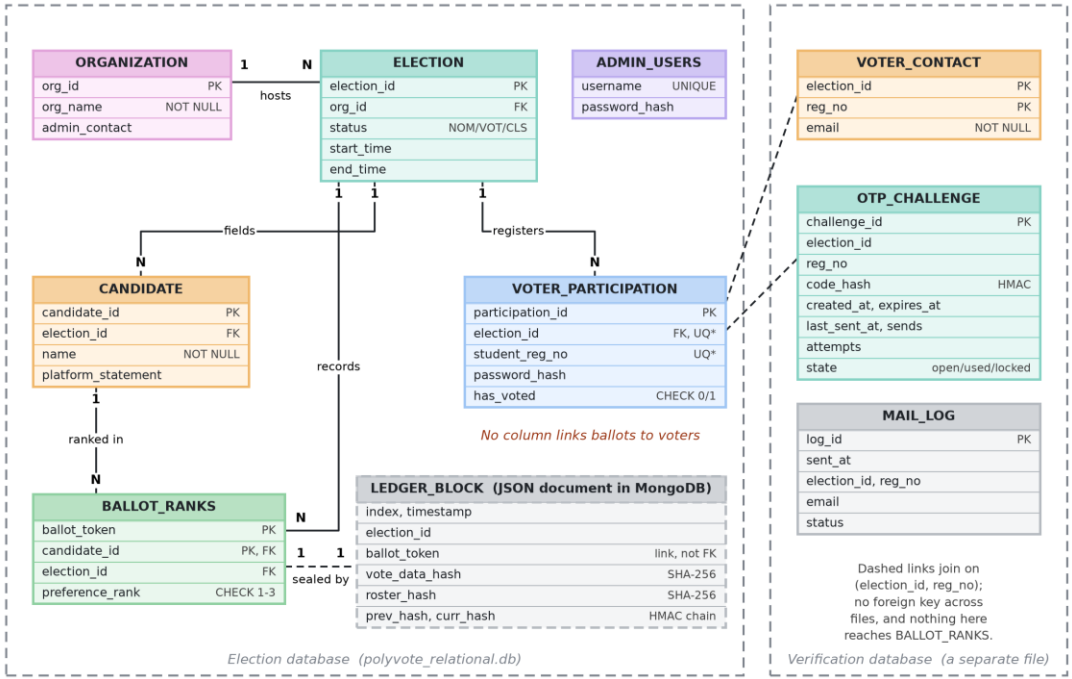
FINAL ARCHITECTURE

Three stores, separated by what each is trusted with



- **Election DB:** roster, candidates, anonymous ballots.
- **Verification DB:** emails and one-time codes, a separate file, never joined to ballots.
- **Ledger:** MongoDB Atlas, HMAC chain.
- **Secrets:** in .env, outside every store.
- **An administrator of one store cannot touch the others.**

Data model: election, ledger and verification stores

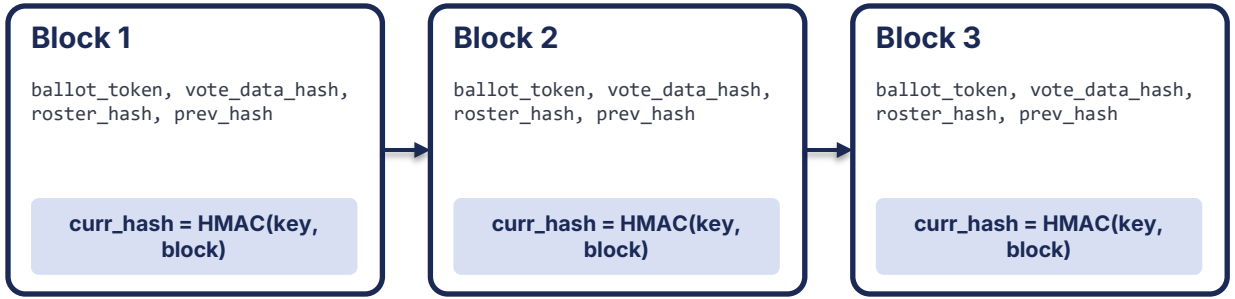


Solid line: foreign key. Dashed: logical link to another store (not an FK). PK primary key, FK foreign key, UQ unique, CHECK check rule.
* UNIQUE (reg_no, election_id). A trigger blocks marking a voter as voted twice.

- Shared multi-tenant schema, scoped by election.
- No column links a ballot to a voter.
- Candidate roster hash sealed in the ledger.
- Emails live in a different file; no key reaches ballots.
- Trigger, CHECK, UNIQUE, FK enforce the rules.

KEYED AUDIT LEDGER

Keyed chain: rebuilding it needs the secret key



`prev_hash` links each block to the one before it.

- **Review 2:** curr_hash was plain SHA-256. An attacker who can write the store can edit a ballot and recompute every later hash.
- **Final:** curr_hash is HMAC-SHA-256 under LEDGER_KEY, held outside the database and the ledger store. Without the key the rebuilt chain fails verification.
- **Roster hash:** every block also seals the candidate list, so renaming candidates after voting is detected.
- **Backward compatible:** blocks written before the key existed still verify (LEDGER_LEGACY_UNTIL).
- **Limit:** whoever holds the key can still forge blocks; that needs an external witness (future work).

INTEGRITY SWEEP

Five rules compare the database with the ledger

Rule	Detects	How	New?
1 Modified	A ballot whose ranks changed	Recompute ranks hash; compare with the receipt	Review 2
2 Deleted	A sealed ballot missing from the database	Receipt exists with no ballot rows	Review 2
3 Injected	A ballot with no receipt	Ballot rows exist with no block	New
4 Roster changed	Candidate names altered after voting	Current roster hash differs from the sealed hash	New
5 Count mismatch	Flags, ballots, receipts disagree	Compare the three counts for the election	New

Plus the chain check: recompute every block's link and hash across the whole ledger. The chain check finds edits inside the ledger; the sweep finds edits to the database.

MONGODB ATLAS LEDGER

A real document store with its own concurrency control

- **Collection:** each block is one document in polyvote.ledger.
- **Unique index on block index:** two writers cannot both claim the next slot, so the chain cannot fork.
- **Append cost:** reads only the last block and inserts one document, so it does not grow with the chain.
- **Fallback:** no connection string, or Atlas unreachable, and it uses the local file; /api/health shows which.
- **Migration:** a script copies the old blocks and verifies the chain; it refuses to overwrite a non-empty collection.
- **Secrets:** the connection string lives in .env, never in the code or the repository.

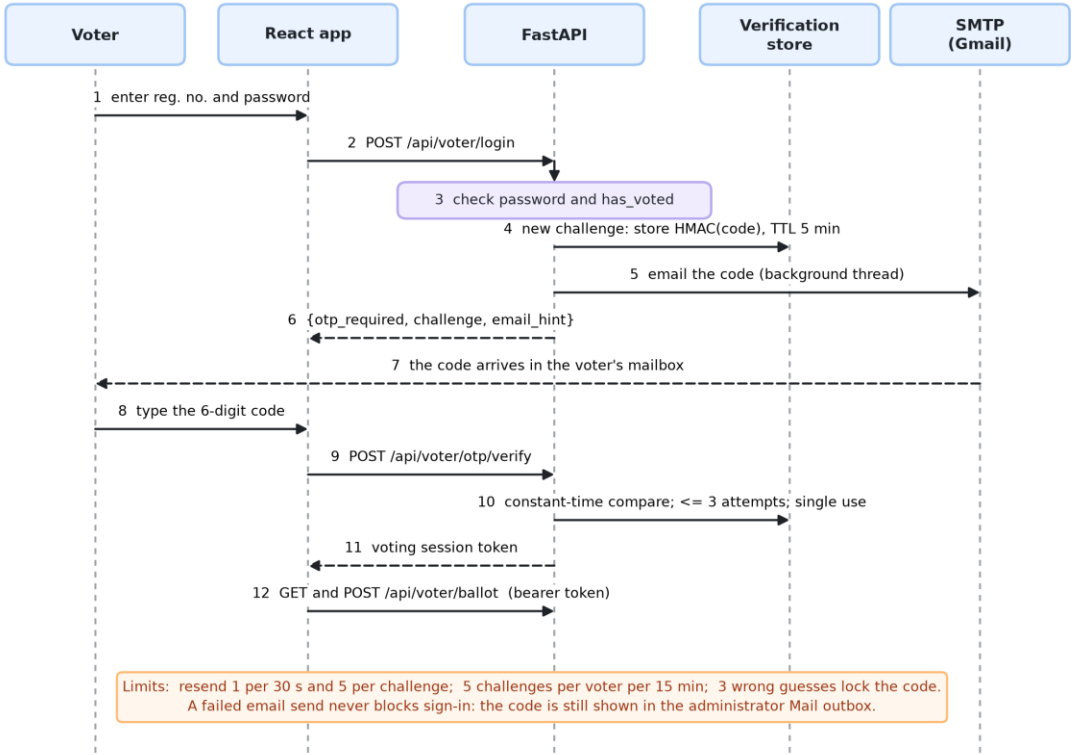
Live health check

```
{"backend":"mongodb",
 "detail":"mongodb
   (polyvote.ledger)",
 "sealing":"hmac-sha256"}
```

Editing any block in Atlas (for example a vote_data_hash) makes the Audit ledger page show "Data tampered at block N".

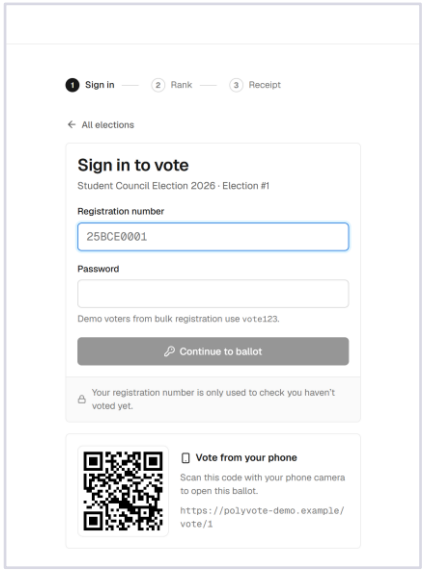
EMAIL ONE-TIME CODE

A correct password no longer opens a ballot on its own

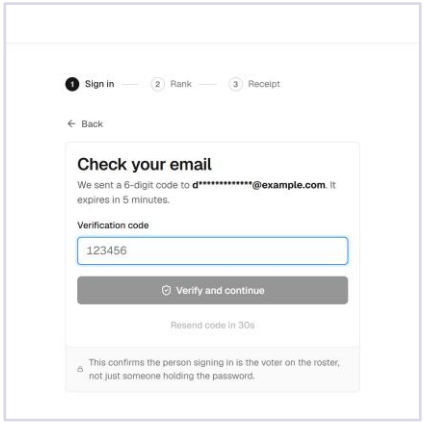


THE VOTER EXPERIENCE

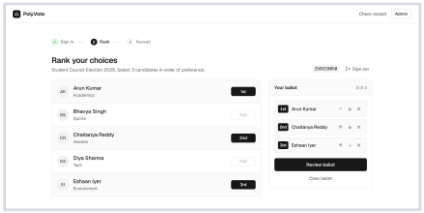
Sign in, confirm by code, rank, and keep a receipt



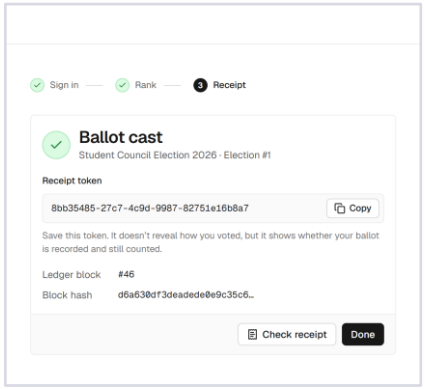
1 Sign in (QR for phones)



2 Enter the emailed code

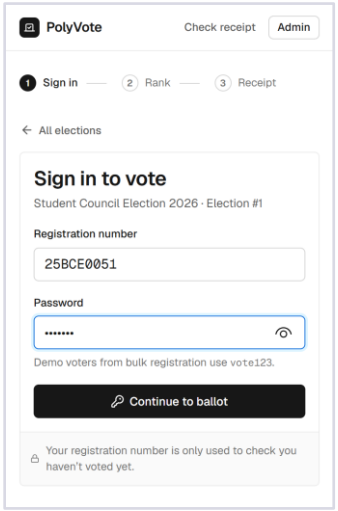


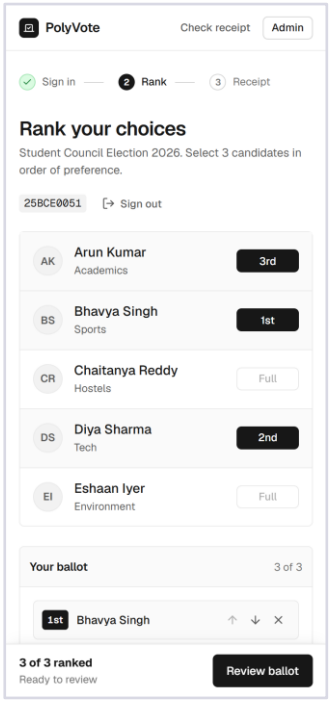
3 Rank three choices



4 Receipt token

Phone voting, a public link, and an honest account





- **How:** a Cloudflare quick tunnel gives a temporary public HTTPS address; the vote page shows a QR code for it.
- **Works from any network;** tested casting a real ballot from a phone.
- **What it is not:** cloud deployment. The laptop must stay on, and the address changes when the tunnel restarts.
- **Why we say so:** a true cloud platform with Oracle is the next step.
- **Safe by default:** secrets stay in .env; the repository holds only .env.example.

LIVE DEMO · SAMPLE POLYVOTE

The same system with email verification switched on

1

Admin

Voters have emails; Mail outbox open

2

Sign in

Password right, still no ballot

3

Code

Read it from the outbox or inbox

4

Vote

Rank, cast, get the receipt

5

Seal

Receipt check; new block in Atlas

6

Tamper

DB Studio edit; sweep names it

7

Ledger

Edit a block in Atlas; chain breaks

Safety net

Every code also appears in the outbox, so a slow email cannot stall the demo.

Separate copy: its own database, its own Atlas collection, port 8001; START_DEMO resets it in about 20 seconds.

HOW WE EVALUATED

47 executable scenarios, old design versus final

7

DB prevention

Statements the schema should reject

13

API abuse

Wrong password, injection, replay, races

13

Tampering

Edits by someone with storage access

14

One-time code

Replay, expiry, limits, failure

3

Benchmark sizes

100, 1,000, 10,000 ballots

- A throwaway copy of the app is driven through the real API; a "rogue administrator" edits the database and ledger files directly.
- Baseline: the Review 2 design, checked out and run unchanged. Attacker scripts never receive the ledger key.
- Outcome per attack: detected by the sweep or chain, visible only in counters, or not detected. Benchmarks are medians of three runs.

PREVENTION RESULTS

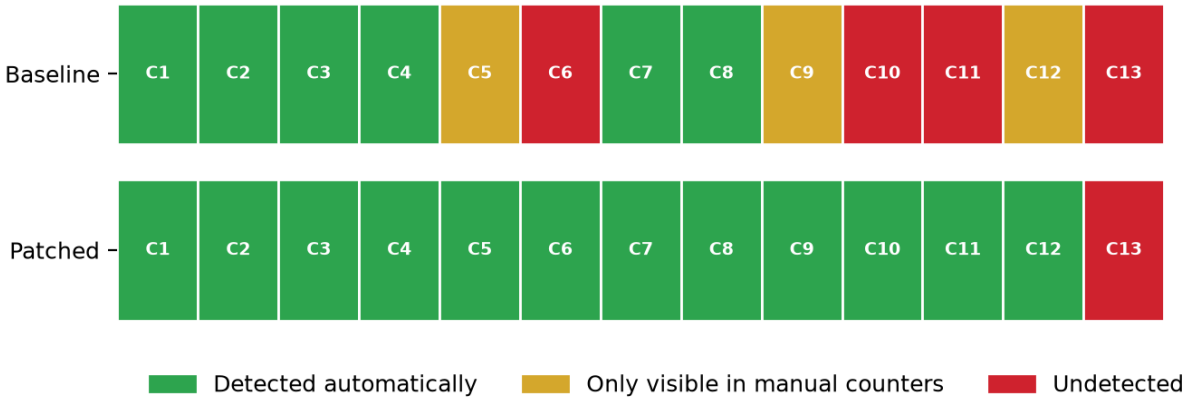
Prevention results: one gap taught us something

Attempt	Stopped by	Result
Second vote by the same voter	Trigger	Prevented
Rank 4 on a 1–3 ballot	CHECK	Prevented
Same candidate ranked twice	Composite key	Prevented
Duplicate roster entry	UNIQUE	Prevented
Wrong password, SQL injection, replayed session	API	Prevented
20 simultaneous casts by one voter	Lock + transaction	Exactly one accepted
Nonexistent candidate from a default console	FK (off by default in SQLite)	Not prevented

The gap: SQLite enforces foreign keys only on connections that enable it. The application does; a console with default settings does not. A server database removes this, which is part of the Oracle move.

TAMPERING RESULTS

Automatic detection rises from 6 of 13 to 12 of 13



6/13

Review 2 design: detected automatically

12/13

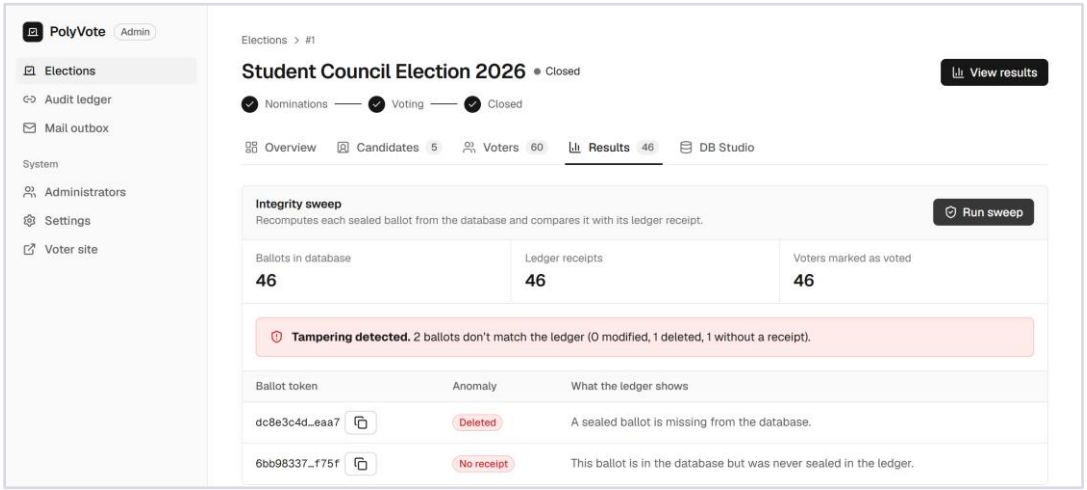
Final design: detected automatically

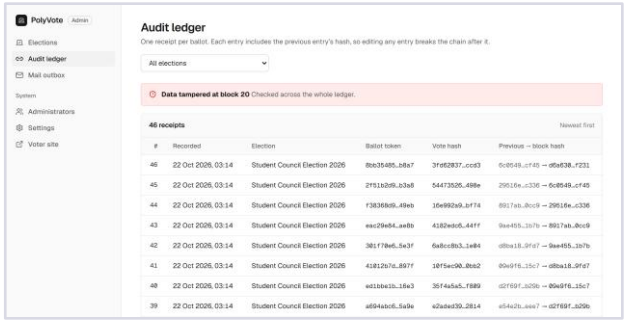
1

Still undetected: roster injection (C13)

SWEEP IN ACTION

A deletion and an injection hide in the counts





- **Left: counters read 46 / 46 / 46, yet the sweep reports 1 deleted and 1 without a receipt.**
- **Above: one edited block breaks the chain.**

WHY THE KEY MATTERS

Six scenarios the Review 2 design missed

ID	Scenario	Review 2	Final	What closed it
C5	Inject a ballot with no receipt	Counters only	Detected	Injected-ballot rule
C6	Ballot stuffing: inject, forge receipt, set a flag	Not detected	Detected	Keyed chain
C9	Truncate the last ledger block	Counters only	Detected	Reconciliation rule
C10	Alter a ballot and rebuild the whole chain	Not detected	Detected	Keyed chain
C11	Swap two candidates' names after voting	Not detected	Detected	Sealed roster hash
C12	Reset a voted flag, vote again	Counters only	Detected	Reconciliation rule

Key point: C6 and C10 defeat any chain that is only hashed, because the attacker recomputes the hashes. With the HMAC key they fail at the first block the attacker cannot compute.

ONE-TIME CODE TESTS

14 of 14 pass, and the suite caught a real defect on the way

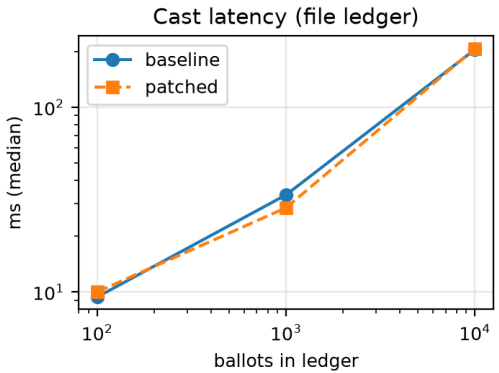
- Password alone gives no session; the challenge id is not a token.
- Right code opens the ballot; a used code cannot be replayed.
- Three wrong codes lock the challenge, even if the right code follows.
- Expired codes, rapid resends and a sixth code are refused.
- No email on file: clear refusal.
- Plaintext codes are absent from the database.
- A failed mail server does not block sign-in.
- No email column exists in the election database.

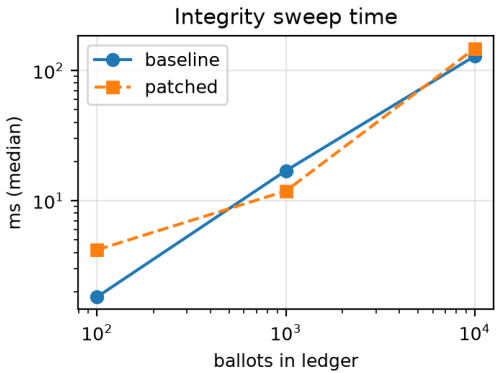
The defect

Test F4 first failed. The wrong-guess counter was saved inside a database context, and the request then raised an error. SQLite rolls back when an exception leaves the context, so every failed guess was recorded and then undone: unlimited guessing. The fix commits state before the error propagates. Review would not have found it; the test did.

PERFORMANCE

Checks are cheap; the file ledger is the one that grows





- Sweep at 10,000 ballots: 147 ms, about 15% slower than before.
- File ledger: a cast grows from 9 to 205 ms because each append rewrites the file.
- Atlas: about 147 ms at any size (network round trip).
- The file is faster when small; Atlas wins on flat growth, concurrency control and living off-machine.

Ballots	Cast ms (file)	Sweep ms	Atlas append ms
100	9	4	136
1,000	33	12	147
10,000	205	147	139

Coverage matrix: who documents which feature

System	F1	F2	F3	F4	F5	F6	F7	F8	F9
Helios	No	–	–	Part	Part	–	–	Yes	Part
Belenios	Part	Part	–	Part	Part	Part	–	Yes	Part
OpaVote	Yes	Part	–	–	–	Part	–	–	Part
ElectionBuddy	Part	Part	–	Part	Part	Part	Part	–	Part
TapVoter	Part	Part	–	Part	Part	Part	Part	Yes	Part
BetterVoting	Part	Yes	–	Part	Part	Part	Part	Yes	Yes
Google Forms	–	–	–	Part	–	No	–	–	Part
FlexyVotes	Yes	Yes	Yes	Part	Part	Yes	No	Yes	Part
VeriVote	–	–	Part	Part	Part	Part	–	Yes	Part
SecureVote	–	No	–	–	Part	Yes	–	–	Part
PolyVote (our tests)	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes

Yes documented

Part partly

No absent

– not documented

33 systems read, 11 shown. F1 ranked counting · F2 many organisations · F3 database rules · F4 unlinked ballots · F5 keyed ledger and sweep · F6 email OTP · F7 QR voting · F8 receipt · F9 easy to run. Other rows: documentation we read. F5 is judged strictly (keyed chain, separate store, reconciling sweep, published attack results), so most systems score Part.

Database rules and unlinked ballots

System	What its own page, README or paper documents	Match
FlexyVotes (GitHub)	Unique indexes and append-only triggers that block even raw SQL; ballot rows carry no voter key. Voters and encrypted emails share one PostgreSQL database.	F3 Yes · F4 Part
VeriVote (GitHub)	Ballots stored without voter identity; triggers, CHECK and foreign-key rules are not documented.	F3 Part · F4 Part
CIVS (Cornell)	Voter emails are erased once the election begins; ballots are stored under a hash of the receipt.	F3 – · F4 Part
Helios, Belenios	Secrecy comes from encryption, not from a schema with no linking column; ballots sit on a public ballot box.	F3 – · F4 Part
Democracy Live (USENIX 2021)	The authors report voter identity, selections and a browser fingerprint sent to the vendor.	F4 No

PolyVote: 6 of 7 direct SQL attempts refused by the engine, 13 of 13 API abuses refused, exactly 1 of 20 simultaneous casts accepted. An automated check finds no column shared by ballots and voters; emails live in a separate file.

Verdict: FlexyVotes is stronger on database rules (append-only triggers). Our known gap is the one attempt not refused: SQLite foreign keys apply per connection.

Tamper-evident ledgers: where a keyed chain differs

System	What its own page, README or paper documents	Match
FlexyVotes (GitHub)	SHA-256 chain per organisation inside PostgreSQL, verified every 6 hours; Merkle bulletin board; 204 tests, no published attack table.	F5 Part
VeriVote (GitHub)	SHA-256 chain in MongoDB with an audit endpoint and a manual tamper demo; no HMAC key, no reconciliation.	F5 Part
SecureVote (GitHub)	Hash-linked ledger with hashed one-time codes; no ranking; a single instance by design.	F5 Part
BetterVoting (GitHub)	Audit log covers administrator actions (sharing a link, revealing a ballot link), not a chain of ballots.	F5 Part
Helios, Belenios	A public, cryptographically verifiable ballot box: a stronger guarantee by a different mechanism.	F5 Part
Voatz (USENIX 2020)	Claims a blockchain; the analysis found no public formal description and reported flaws.	F5 Part

PolyVote: an HMAC-keyed chain in a separate MongoDB store, a five-rule sweep and a published attack suite. 12 of 13 attacks caught against 6 of 13 for the plain-chain design, the same in 3 runs; C6 and C10 only fail with the key.

Verdict: better than a plain chain against someone who can write to the database. Not a match for cryptographic verification, and whoever holds the key can still forge.

Emailed codes exist elsewhere; so does QR sharing

System	What its own page, README or paper documents	Match
FlexyVotes (GitHub)	Keyed-hash one-time code with expiry, attempt limit and hourly rate limit; QR only for event ticket check-in.	F6 Yes · F7 No
SecureVote (GitHub)	Hashed one-time code with expiry and attempt limits.	F6 Yes
VeriVote (GitHub)	Email one-time code with rate limits.	F6 Part
BetterVoting	A unique emailed link per voter; docs mention no one-time code or 2FA; a QR code shares the election link.	F6 Part · F7 Part
TapVoter	Single-use key handed out by QR poster or email; no emailed second-factor code documented.	F6 Part · F7 Part
Belenios	Default voter login is an emailed short code at voting time; hashing, expiry and limits not documented.	F6 Part

PolyVote: password plus an emailed six-digit code (hashed, 5 minutes, single use, 3 guesses, resend limits). 14 of 14 tests pass, and one real defect (unlimited guesses) was found and fixed. A QR code on the vote page opens the ballot on any phone; a real ballot was cast from one.

Verdict: F6 alone is not unique: FlexyVotes and SecureVote have it too. What we did not find documented is a scan-to-vote QR flow together with an emailed code.

COMPARATIVE STUDY · CLOSEST PROJECTS

The three closest projects, and what each lacks

Closest project (source)	What it has	What we found it does not document
FlexyVotes, Django and PostgreSQL (github.com/deOpty/flexyvotes)	Many organisations; instant-runoff and STV; triggers that block raw SQL; ballots with no voter key; hash-chained audit log; hashed single-use email OTP	QR scan-to-vote; an HMAC-keyed chain; a separate document store and email database; a reconciling sweep; published attack results
VeriVote, Node, React and MongoDB (github.com/9022574361/Online_Voting_System)	SHA-256 chain in MongoDB with an audit endpoint; ballots without voter identity; email OTP with rate limits; receipt hash	Ranked counting; many organisations; database rules; HMAC key; reconciliation; automated attack suite
BetterVoting (github.com/Equal-Vote/bettervoting)	Ranked-choice voting; teams; QR sharing; receipts; free and open source (AGPL)	Database-enforced rules; a ledger of ballots (its audit log covers admin actions); one-time-code factor; reconciliation and attack tests

What we claim: every single feature exists somewhere, so PolyVote is not the first or the only one. We found no free small-election tool documenting all of them together. **What we do not claim:** to be more secure than Helios, Belenios or ElectionGuard. Limits: GitHub code search and non-English repositories were not covered.

COMPARATIVE STUDY · VERDICT

Best for small elections, not a Helios replacement

PolyVote IS A GOOD FIT WHEN

- A club, council or class needs ranked-choice results.
- Someone must be able to run it in an afternoon, free.
- You need proof that edits and deletions are caught.
- Voters should confirm themselves by email and vote from a phone.

CHOOSE HELIOS / BELENIOS WHEN

- The administrator itself cannot be trusted.
- Voters must verify their own ballot content.
- Coercion or vote selling is a real threat.
- A public cryptographic audit is required.

Our claim: for small elections, PolyVote adds database-level rules and tested tamper detection to an easy setup. It is not the strongest on verifiability.

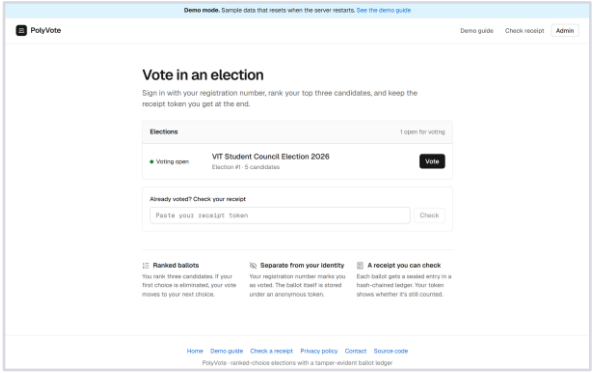
REVIEW 2 DESIGN VERSUS FINAL DESIGN

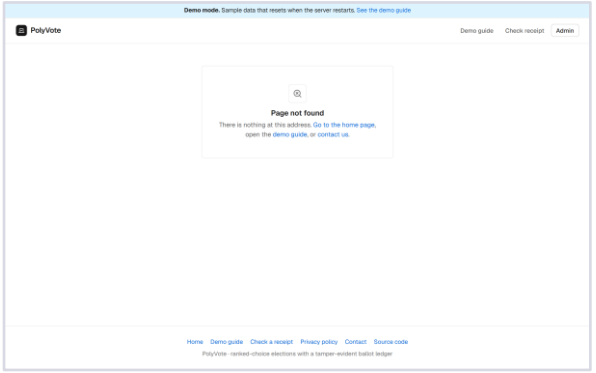
What improved, measured

Measure	Review 2	Final
Tampering detected automatically	6 of 13	12 of 13
Forged receipt or chain rewrite	Undetected	Detected (key)
Candidate renaming	Undetected	Detected (sealed roster)
Voter authentication	Password	Password + emailed code
Ledger storage	Local file	MongoDB Atlas
Ledger append at 10,000 ballots	205 ms (file, whole cast)	~147 ms (Atlas, flat)
Sweep at 10,000 ballots	129 ms	147 ms
Access	Local only	Public link + QR

Website launch checklist: 17 pass, 4 N/A, 0 fail

- PASS Server-side rendering (pre-rendered shell)
- N/A HTTPS and mobile viewport
- PASS robots.txt
- PASS sitemap.xml
- PASS Unique title tags
- PASS Meta descriptions
- PASS One H1 per page
- PASS Canonical tags
- PASS Open Graph images
- PASS Custom 404 page
- PASS 301 redirects
- PASS Alt text on images
- N/A WebP images
- PASS Lazy loading
- PASS Organization schema
- PASS Breadcrumbs
- PASS Internal links
- N/A Google Analytics
- N/A Search Console
- PASS Privacy policy
- PASS Working contact form





N/A: HTTPS (a local run; the host provides it), WebP (no raster images in the pages), Google Analytics and Search Console (need the owner's own IDs).

Privacy and ethics: what is stored, and where

Data	Where it lives	Stored as
Registration number	Election database	Roster entry
Password	Election database	SHA-256 hash only
Email address	Verification database (separate file)	Never stored with ballots
One-time code	Verification database	Keyed hash only; 5 min, one use
Your ballot	Election database	Random token; no voter column
Receipt block	Audit ledger	Token and hash of the ranks
Contact message	Verification database	Admins only

The ballot table and the voter table share no column, so the database cannot say who cast which ballot. A person with direct file access could read ballots, but not link them to people.

ETHICS AND LIMITS

- **Sample data:** the demo uses sample voters and resets on restart.
- **Officers see more:** the roster, emails and an outbox that lists codes for ten minutes. We kept it so a demo never stalls; a real election should switch it off.
- **Receipts:** prove a ballot is counted, not its content.
- **No tracking:** no tracking cookies; analytics is off unless the operator enables it.
- **Not for binding votes:** no independent review yet; passwords use plain SHA-256.

LIMITATIONS AND FUTURE WORK

What we do not claim, and what comes next

Limitation	Direction
Insider with key and both stores can forge (A6)	Publish the chain head to an external witness; managed secret store
Roster injection is not detected (A7)	Committed or signed roster, checked at sweep time
Not end-to-end verifiable; receipt shows recorded, not content	Cast-as-intended mechanism, or a verifiable protocol if needed
Passwords stored as plain SHA-256; default admin login	Salted, slow hashing; forced first-login change
Email is a weak second factor	Authenticator app (TOTP) or printed one-time slip
IRV ties broken arbitrarily; no statistical audit	Published tie rule; risk-limiting audit
SQLite foreign keys are per connection; single node	Oracle or PostgreSQL with PL/SQL, roles; cloud platform

GLOSSARY

Glossary in plain words

Term	In plain words
Hash	A fixed-length fingerprint of data. Change one character and the fingerprint changes completely.
HMAC	A hash made with a secret key, so only someone holding the key can produce a valid one.
Hash chain	Each record stores the previous record's hash, so editing one breaks every later link.
Ledger, block	The append-only list of receipts; one block for each ballot.
Instant-runoff	Rank candidates; last place drops out each round and its ballots move on, until someone has a majority.
OTP	One-time password: a short code that works once and expires.

Term	In plain words
Roster hash	A fingerprint of the candidate list, sealed into every block.
Receipt token	A random code that shows a ballot is recorded. It does not show the vote.
Trigger, CHECK, UNIQUE	Rules the database enforces itself, even against direct SQL.
Integrity sweep	Recomputes the election from the database and compares it with the ledger.
Polyglot persistence	Different kinds of database for different jobs: SQLite and MongoDB here.
Multi-tenant	Many organisations share one install, each with its own elections.

EVIDENCE

Evidence: where every claim can be checked

Claim in this deck	How it was produced	Where to look
Tampering caught: 6 of 13 (old), 12 of 13 (final)	Executable suite, 3 identical runs	experiments/results/summary.json, detection.png
Database refuses bad votes (6 of 7; 13 of 13; 1 of 20)	Direct SQL and API tests	experiments/run_experiments.py
14 of 14 one-time-code tests	Tests F1 to F14 on a throwaway copy	experiments/run_experiments.py
Sweep +15% at 10,000 ballots; Atlas append flat	Benchmarks, median of 3 runs	bench.png, mongo_bench.json
Website checklist 17 / 4 / 0	Automated check of 21 items	experiments/web_checklist.py, web_checklist.json
What 33 other systems offer	Their own pages, READMEs, papers; a URL for every cell	docs/review3/uniqueness_evidence.md
Closest projects and their gaps	FlexyVotes code and docs read directly	docs/review3/uniqueness_evidence.md

Search method: 23 web queries, 20 GitHub repository searches, 33 systems, on 23 October 2026. **Not covered:** GitHub code search, non-English and private repositories, commercial internals.

INDIA ONLY · NIRF TOP-100 HOSTS · OPEN TODAY

India only, NIRF top-100 hosts, cheapest first

#	Venue	Host university (NIRF 2025 rank)	Submit by	Student fee, INR	Held	Indexing
1	SCES-2027	MNNIT Allahabad, Prayagraj (Engineering 62)	15 Nov 2026*	5,900 incl. GST	12-14 Mar 2027	Claimed
2	IEEE COEP CON 2027	COEP Technological Univ., Pune (Engineering 90)	31 Mar 2027	6,000†	4-6 Jun 2027	Claimed
3	IEEE SANKALP 2027	ABV-IIITM, Gwalior (Engineering 96)	30 Nov 2026	7,000	9-11 Apr 2027	IEEE record #73342 shown
4	ICESICSC-2027	KIIT, Bhubaneswar (Overall 27)	30 Oct 2026	9,000	20-21 Feb 2027	Claimed
5	Confluence 2027	Amity University, Noida (Overall 37)	4 Dec 2026	9,440 incl. GST	21-22 Jan 2027	IEEE record #71526 shown
6	INDIACom-2027	BVICAM, Bharati Vidyapeeth, New Delhi (parent only, University 59)	7 Dec 2026	9,440 incl. GST	7-9 Apr 2027	Claimed
7	i-PACT 2027	VIT, Vellore (Overall 21)	31 May 2027	12,000	2-3 Sep 2027	Claimed
8	ICICC-2027	Shaheed Sukhdev College, Univ. of Delhi (parent only, Overall 15)	30 Oct 2026	13,000	5-6 Feb 2027	Claimed (LNNS "approved")

**Page marks the date tentative. †Fee read from a flattened table. "IEEE record shown" means a record number is on the official page, not checked on the IEEE site; nothing here is confirmed for IEEE Xplore. Also qualify, but off-topic or no published fee: ICSEAMS (Amrita), ICCIDS (SSN), PerCom work-in-progress (BITS Pilani Goa), NCC (IISc). Dropped, host not in the top 100: ICRCICN, IEM-ICDC, iConCISE, ICPCSN.*

RECOMMENDED SUBMISSION ORDER

One venue at a time: dual submission is not allowed

Step	Venue	Host university	Submit by	Why this order
1	IEEE SANKALP 2027	ABV-IIITM Gwalior	30 Nov 2026	IEEE record number shown; security and distributed-database tracks; INR 7,000
2	Confluence 2027	Amity University, Noida	4 Dec 2026	IEEE record number shown; INR 9,440; fast decision
3	ICESICSC-2027	KIIT, Bhubaneswar	30 Oct 2026	Closes in 7 days; only if the paper already fits their template
4	INDIACom-2027	BVICAM, Bharati Vidyapeeth	7 Dec 2026	Borderline host: only the parent university is ranked
5	i-PACT 2027	VIT, Vellore	31 May 2027	Our own campus, but a power-engineering conference

Before submitting: check the IEEE site for each record number, recheck the deadline and the NIRF edition (a 2026 list may be published), remove author names where double-blind, and follow each venue's AI-disclosure rule. Journals cost USD 2,000 to 3,000 in fees: only as a later extended version.

PROJECT CONCLUSION

Planned versus delivered

Item planned at Reviews 1 and 2	Status	Note
Multi-tenant normalised schema, views, trigger	Delivered	On SQLite
Instant-runoff tally and admin console	Delivered	Round-by-round results
Hash-chained receipts and integrity sweep	Delivered, extended	Keyed chain, five sweep rules
MongoDB ledger	Delivered	Atlas, with a file fallback
Second factor for the voter	Delivered	Emailed one-time code
Public hosting and phone voting	Delivered	Public link, QR code, free hosted demo configured
Comparative study and evaluation	Delivered	47 test scenarios; 33 systems read
True cloud platform	Partly	Free-tier hosting only; no managed cloud database
Oracle with PL/SQL procedures and DCL roles	Left out	SQLite kept; the path is on the next slide
POSITION entity	Left out	Deferred

Why SQLite, and how Oracle is possible

WHY WE CHOSE SQLITE

- **Nothing to install:** one file, no server; anyone runs it on a laptop or free hosting with one command.
- **Repeatable tests:** the evaluation copies the database file into a throwaway sandbox, so all 47 scenarios rerun the same way.
- **Cost and time:** Oracle needs a licence or cloud account and set-up that a semester project and a hotspot demo cannot rely on.
- **Trade-off we accept:** foreign keys only where switched on, one node, no roles.

HOW ORACLE WOULD FIT

- **Schema:** the 5 tables, 1 trigger and 2 views sit in one file; turn AUTOINCREMENT into identity columns and the trigger into PL/SQL.
- **Driver:** database calls are in a handful of modules; swap sqlite3 for python-oracledb and change the placeholders.
- **Add what SQLite lacks:** GRANT-based roles and PL/SQL procedures; foreign keys always on, which closes the one gap our tests found.
- **Unchanged:** the MongoDB ledger, the one-time-code module and the React app.

This is the planned path, not a result: we have not run it on Oracle.

PROJECT CONCLUSION · REVIEW 3

Thank You

Questions and discussion

Source code and documents

github.com/Tharakeshtj/polyvote-election-platform